CS228: Introduction to Deep Learning

Lecture 10: training techniques

Instructor: Zhouxing Shi

Updates

- Assignment 1 and project proposal will both be due on Tuesday
- Assignment 1:
  - You may leverage torch.nn.functional.one_hot to simply things a bit
  - Test accuracy expectation: best accuracy across all the epochs
  - Partial credit will be given for test accuracy below 98%
- Fixed submission for the project proposal
- Reserve your midterm on PrairieTest if you haven’t done so

Recap: optimizers using gradients

Plain gradient descent:

$$\mathbf{W}^{(t)} \leftarrow \mathbf{W}^{(t-1)} - \alpha \nabla_{\mathbf{W}^{(t-1)}} f(\mathbf{W}^{(t-1)})$$

Adam (RMSProp + Momentum):

$$m_i^{(t)} = \beta_1 m_i^{(t-1)} + (1-\beta_1) g_i^{(t-1)}$$

$$v_i^{(t)} = \beta_2 v_i^{(t-1)} + (1-\beta_2) \left(g_i^{(t-1)}\right)^2$$

$$\hat{m}_i^{(t)} = \frac{m_i^{(t)}}{1-\beta_1^t} \qquad \hat{v}_i^{(t)} = \frac{v_i^{(t)}}{1-\beta_2^t}$$

$$w_i^{(t)} \leftarrow w_i^{(t-1)} - \frac{\alpha}{\sqrt{\hat{v}_i^{(t)}} + \epsilon} \hat{m}_i^{(t)}$$

Gradient estimation

- We technically need gradients for the loss function on the entire training dataset $\{(\mathbf{x}^{(1)}, y^{(1)}), (\mathbf{x}^{(2)}, y^{(2)}), \cdots, (\mathbf{x}^{(n)}, y^{(n)})\}$

$$\nabla f(\mathbf{W}^{(t)}) = \frac{1}{n} \sum_{i=1}^{n} \nabla f^{(i)}(\mathbf{W}^{(t)})$$

$$f^{(i)}(\mathbf{W}^{(t)}) = \ell(h_{\mathbf{W}^{(t)}}(\mathbf{x}^{(i)}), y^{(i)})$$

(We use “h” to denote the model output)

Too slow for large datasets: The model is only updated once after going through the entire dataset

Stochastic Gradient

- Sample a small subset to **estimate** the gradient:

$$\nabla f(\mathbf{W}^{(t)}) \approx \frac{1}{|B|} \sum_{(\mathbf{x},y)\in B} \nabla f^{(\mathbf{x},y)}(\mathbf{W}^{(t)})$$

gradient + zero-mean noise

- Stochastic gradient is an *unbiased estimator* of full gradient:

$$\mathbb{E}\left[\frac{1}{|B|} \sum_{(\mathbf{x},y)\in B} \nabla f^{(\mathbf{x},y)}(\mathbf{W}^{(t)})\right] = \frac{1}{n}\sum_{i=1}^{n} \nabla f^{(i)}(\mathbf{W}^{(t)}) = \nabla f(\mathbf{W}^{(t)})$$

Stochastic gradient descent (SGD)

- Go over the training dataset for multiple **epochs**
- In each epoch, go over the dataset once in small **batches**
- Batch size:
  - Typically set to the **largest** supported by GPU
  - Large values: higher GPU utilization, less variance, fewer batches per epoch
- Typically use advanced SGD optimizers such as Adam

Stationary points

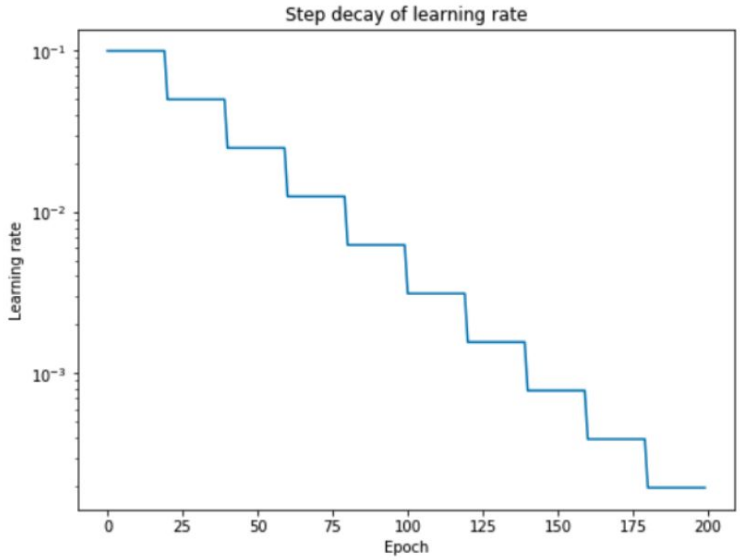
- A stationary point: $\nabla f(\mathbf{W}^*) = \frac{1}{n}\sum_{i=1}^{n} \nabla f^{(i)}(\mathbf{W}^*) = 0$

- Most likely (due to random noise): $\frac{1}{|B|}\sum_{(\mathbf{x},y)\in B} \nabla f^{(\mathbf{x},y)}(\mathbf{W}^*) \neq 0$

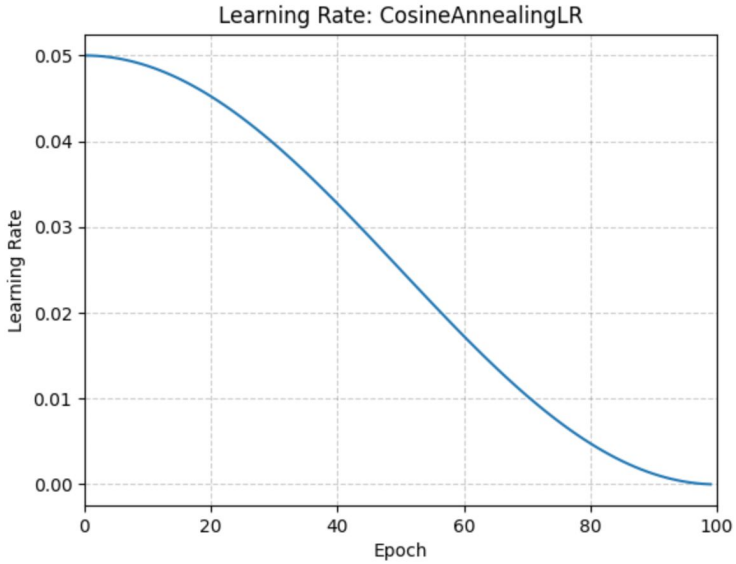
- Saddle point: SGD is better at jumping out of saddle points
- Local minimum: SGD (with a fixed learning rate) will move away from it
- SGD with a fixed learning rate cannot converge to global/local minimum

Learning rate schedule

- For SGD to converge, the learning rate should decrease to 0



Step decay

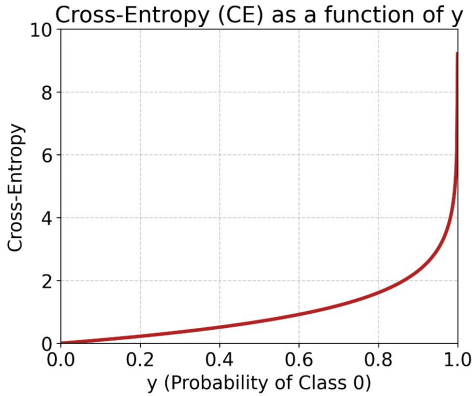


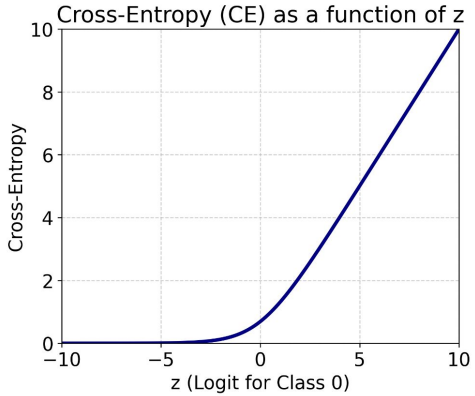
Cosine Annealing

Loss function: CE v.s. L2 for classification

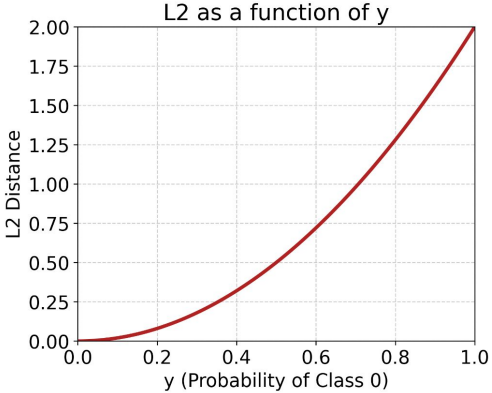
After softmax | Before softmax

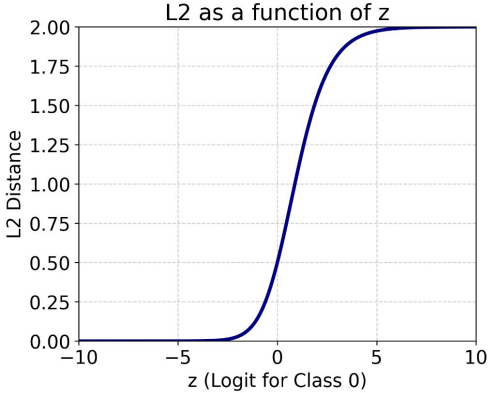
CE





L2





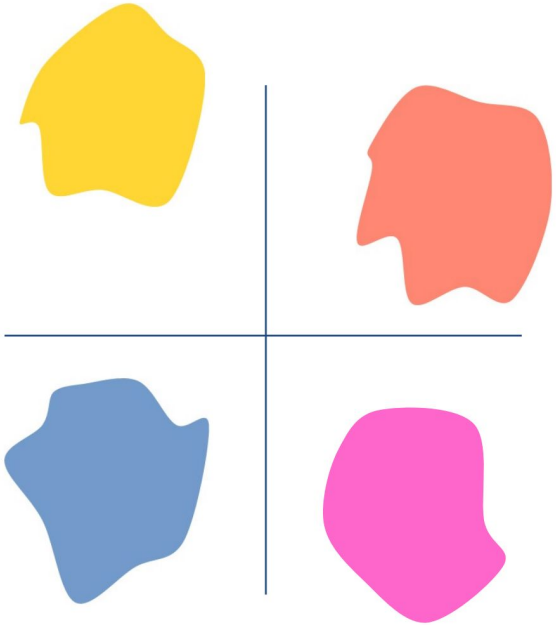
Use CE for classification

L2:

- Tiny gradient for large
- Non-convex

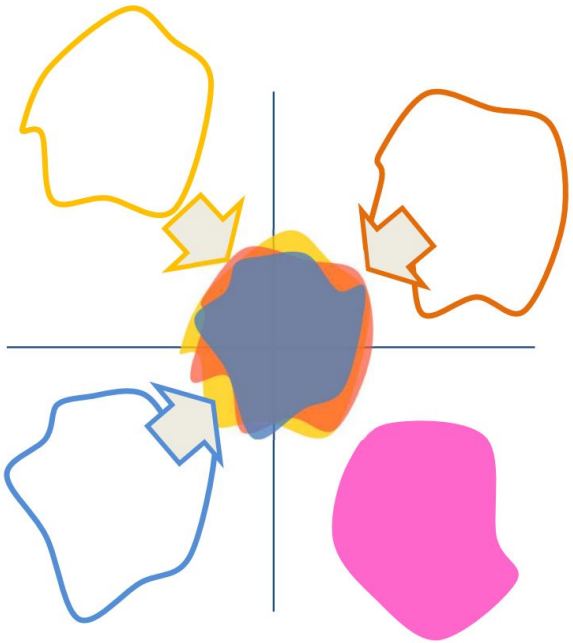
Covariate shifts

- Each batch may have a different distribution in each layer
- But training assumes each batch has a similar distribution



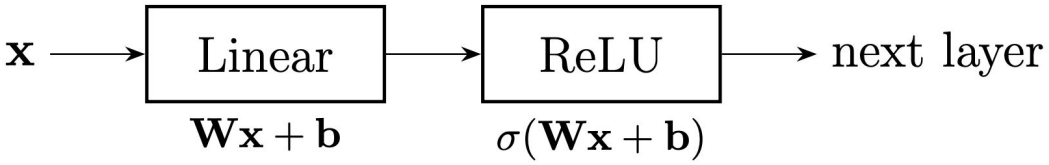
Batch normalization

- Move all the batches to have zero mean and unit variance

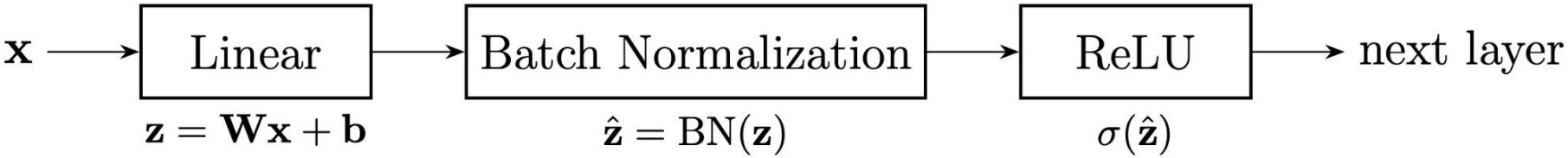


Batch normalization

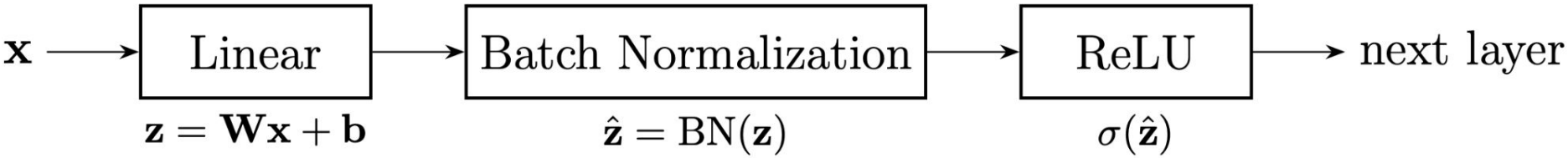
Without batch normalization:



With batch normalization:



Batch normalization



Training: Compute mean and variance per neuron over the batch:

$$\mu_j = \frac{1}{B}\sum_{i=1}^{B} z_j^{(i)} \qquad \sigma_j^2 = \frac{1}{B}\sum_{i=1}^{B}(z_j^{(i)} - \mu_j)^2$$

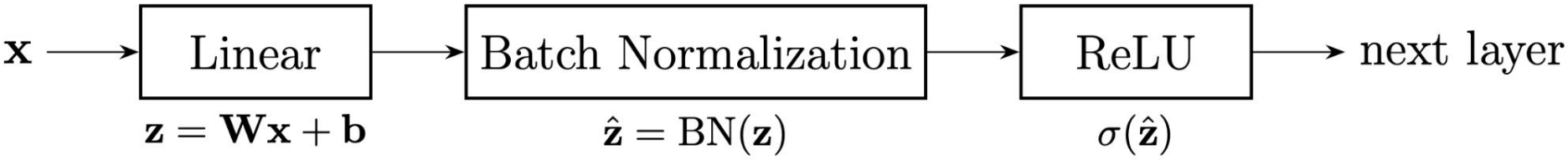
Normalize to zero mean and unit variance:

$$u_j^{(i)} = \frac{z_j^{(i)}}{\sqrt{\sigma_j^2 + \epsilon}}$$

Add learnable shifting and scaling:

$$\hat{z}_j^{(i)} = \gamma u_j^{(i)} + \beta$$

Batch normalization

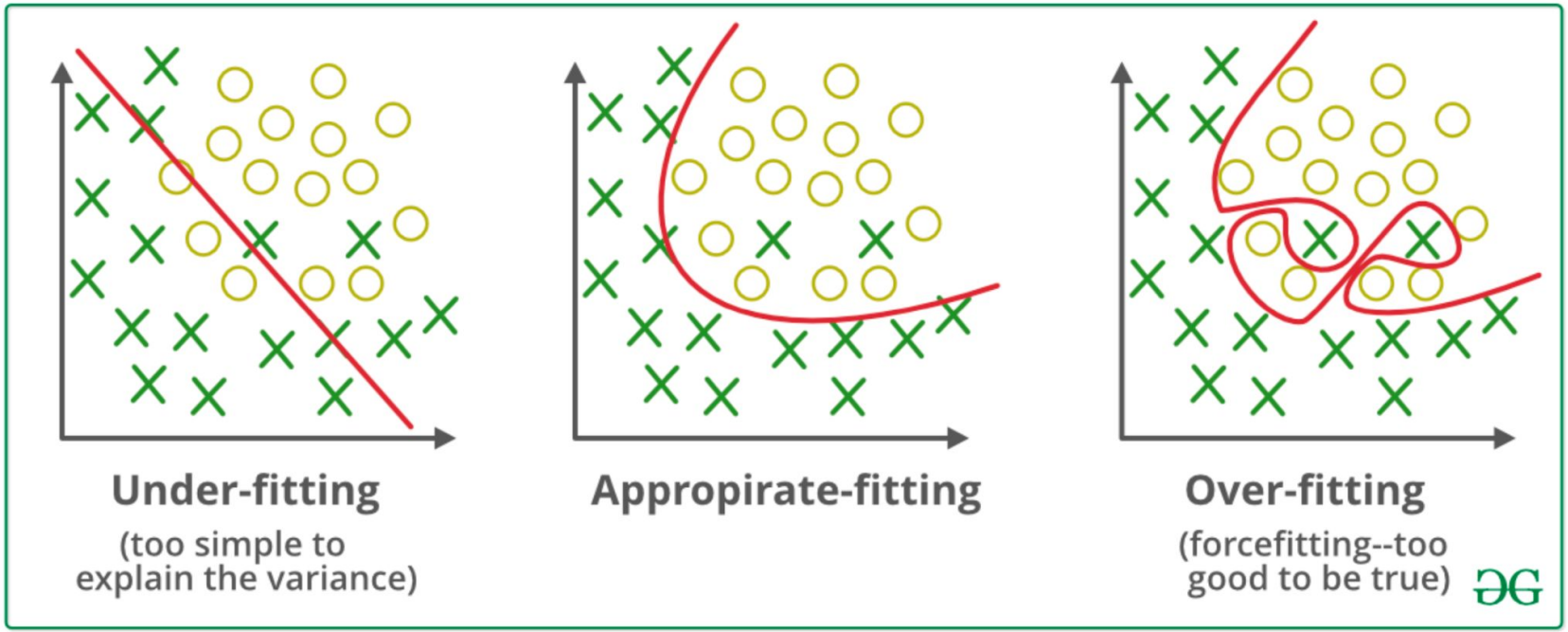


Inference:

- No longer doing training -> covariant shift doesn't matter
- Each individual test example must be handled independently
- Use *running mean* and *running variance* from training (exponential moving average of mean and variance across all steps)

Overfitting

- The model may just learn the training examples with an over-complex function



Regularization

- Restrict the complexity of the learned model
- L2 regularization: extra term for the (Frobenius) norm of weights

$$L = \mathrm{CE} + \lambda \|\mathbf{W}\|_F^2$$

- λ controls the importance of shrinking weights

Weight decay

- Regularization on weights:

$$L = \mathrm{CE} + \lambda \|\mathbf{W}\|_F^2$$

- Viewed as a weight decay (set as a hyperparameter of the optimizer):

$$\begin{aligned}\mathbf{W}^{(t+1)} &= \mathbf{W}^{(t)} - \alpha\left(\nabla f(\mathbf{W}^{(t)}) + 2\lambda \mathbf{W}^{(t)}\right) \\ &= \underbrace{(1 - 2\alpha\lambda)}_{\text{weight decay}} \mathbf{W}^{(t)} - \alpha \nabla f(\mathbf{W}^{(t)})\end{aligned}$$

Dropout

- Regularization by adding noise in training
- Training:
  - Each neuron has probability p of being zeroed
  - Each non-zero neuron is scaled by $1/(1-p)$
- Testing: $p=0$

Training and eval modes

- Batch normalization and dropout require different behaviors during training and inference
- In PyTorch:
  - Set model.train() for training
  - Set model.eval() for evaluation/inference
  - The mode is tracked in model.training